RAPIDS, Spark, and Handling Skew

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE
Director, Centre for Artificial Intelligence Research,
Prof., School of Computer Science & Engineering,
KLE Technological University,
BVB Campus, Vidyanagar,
Hubballi - 580031, Karnataka, India

The Big Picture — the 'Big' Tools for 'Big' Workloads

Dask and Ray (Session 2) cover most Python-native needs. This session is for when you outgrow even those: GPU-accelerated DataFrames, and the petabyte-scale standard, Spark.

THINK OF IT LIKE THIS: If Dask is a fast bicycle and Ray is a versatile van, RAPIDS is a race car (blazing fast, narrow use case) and Spark is a freight train (huge capacity, mature infrastructure, works at a scale nothing else touches).

1 RAPIDS puts pandas on the GPU

Same API, 10-100x faster for the right workload shape.

2 Spark is still the default at petabyte scale

Mature SQL optimiser, lakehouse ecosystem, battle-tested.

3 Data skew is the recurring killer

One overloaded key can turn a 5-minute job into an 8-hour one.

Key Terms Before We Start

cuDF — A GPU-accelerated DataFrame library with a pandas-like API.

Data skew — When one partition/key has far more data than the others, creating a bottleneck.

Executor — A Spark worker process that runs tasks and holds data in memory.

Salting — Adding a random value to a skewed key to spread its rows across more partitions.

Catalyst optimiser — Spark's query planner that rewrites your code for better performance automatically.

RAPIDS cuDF — pandas on GPU

```
# PANDAS (CPU)
import pandas as pd
df =
pd.read_parquet('trades.pa
rquet')
agg =
df.groupby('symbol').agg({
'value':'sum'})
# 80M rows: ~25 sec on 32
cores
```

```
# cuDF (GPU)
import cudf
df =
cudf.read_parquet('trades.
parquet')
agg =
df.groupby('symbol').agg({
'value':'sum'})
# 80M rows: ~0.4 sec on
one A100
```

Same API. ~50x faster on the right workload.

When cuDF wins — and when it doesn't

WINS: column-oriented ops on 1M+ rows fitting in GPU memory.
LOSES: small datasets (<100k rows), heavily branchy Python UDFs, I/O-bound work.

Dask + cuDF — Scale RAPIDS Across GPUs

Single-GPU cuDF is fast. Multi-GPU dask_cudf is faster AND handles datasets bigger than one GPU's memory.

```
from dask_cuda import LocalCUDACluster; import dask_cudf
cluster = LocalCUDACluster(n_workers=8)   # 8 GPUs, one
worker each
df = dask_cudf.read_parquet('s3://huge/')   # looks like
dask.dataframe, runs on cuDF
agg =
df.groupby(['symbol','minute']).value.mean().compute()
```

When it wins

Multi-TB tabular workload, 8+ GPUs, ETL is the bottleneck.

When it doesn't

Small data. Heavy SQL workloads (Spark's optimiser is more mature).

Spark — the Heavy Hitter

Petabyte scale

Battle-tested at Databricks/LinkedIn/Netflix scale.

Mature SQL optimiser

Catalyst rewrites queries automatically for speed.

Lakehouse ecosystem

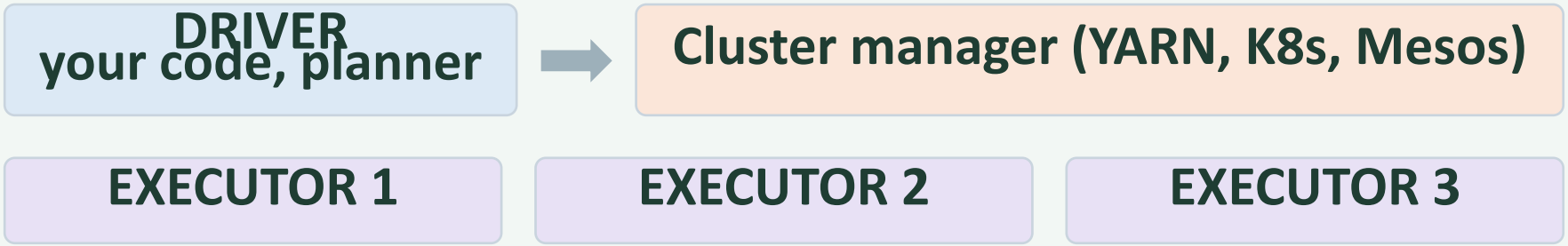
Delta Lake, Iceberg — ACID transactions on object storage.

Multi-language

Scala, Python, R, SQL — same job, same performance.

RULE OF THUMB: if your dataset is in a lakehouse or measured in TB+ daily, you'll meet Spark.

Spark Architecture in One Diagram



Job: One action (count, write). Splits into stages.

Stage: Contiguous tasks with no shuffle between.

Task: Smallest unit — one partition processed by one slot.

Partition: A chunk of the dataset. More partitions = more parallelism.

Data Skew + the Salted-Groupby Pattern

THINK OF IT LIKE THIS: 100M web events, 8 workers. ONE user (a bot) generates 50% of all events. After hash partitioning, ONE worker holds half the data and takes hours while the other 7 idle in minutes.

```
df = df.withColumn('salt', (F.rand()*16).cast('int'))
# 1st agg splits the skewed key across 16 sub-partitions
partial =
df.groupBy('user_id','salt').agg(F.sum('value').alias('ps
um'))
# 2nd agg drops the salt — small, fast, since row count
is already reduced
final =
partial.groupBy('user_id').agg(F.sum('psum').alias('total
'))
```

WHY THIS WORKS: the first groupby balances load across 16 partitions per skewed key. The second is small (few rows after partial aggregation). Trade-off: 2 shuffles instead of 1 — wins big when one key dominates.

Adaptive Query Execution — Automating the Manual Fix

Spark 3.0+ can detect and fix skew automatically at runtime, using real statistics rather than upfront guesses.

```
spark.conf.set('spark.sql.adaptive.enabled', 'true')# on by default in
Spark 3.2+
spark.conf.set('spark.sql.adaptive.skewJoin.enabled', 'true')
spark.conf.set('spark.sql.adaptive.coalescePartitins.enabled', 'true')
```

- AQE re-plans the query AFTER seeing real shuffle statistics
- Skew join handling automatically splits an oversized partition — the runtime equivalent of manual salting
- Still worth knowing the manual pattern: Dask has no AQE equivalent, and older Spark may have it disabled

Common Mistakes & How to Spot Them

Mistake	How it shows up	Fix
Reaching for cuDF on small data	GPU version is SLOWER than pandas	cuDF wins above ~1M rows, not below
Ignoring one slow Spark stage	One task at 30 min, 199 tasks at 30 sec	That's skew — apply salted groupby or enable AQE
Calling .collect() on a huge dataset	The driver runs out of memory and crashes	Write to storage instead of collecting to the driver

Quick Recap — Terms and Concepts

cuDF — GPU pandas — ~50x faster on the right workload, loses on small/branchy data

Spark — The petabyte-scale default — mature SQL optimiser, lakehouse ecosystem

Data skew — One overloaded key/partition bottlenecks the whole job

Salted groupby — Spread a hot key across N sub-partitions, then re-aggregate

AQE — Spark's automatic, runtime version of the salting fix